Date: 01-11-2026

Lab # 07 TO UNDERSTAND HASHING TECHNIQUES

Lab Objective:

- Understand the concept of hashing and hash functions.
- Analyze the performance of hash tables under different scenarios.

Theory:

Hashing is a technique used to map data (keys) to a fixed-size table (hash table) using a function called a **hash function**. It is widely used for efficient data retrieval in scenarios like database indexing, caching, and cryptography.

Components of Hashing

1. **Hash Function**
   A hash function converts a given key into a numerical index within the bounds of the hash table size. A good hash function ensures:
   - Uniform distribution of keys across the table to minimize collisions.
   - Efficiency, so it can handle large datasets quickly.

2. **Hash Table**
   A hash table is an array-like data structure where data is stored at indices calculated by the hash function. ○ Size of the hash table should ideally be a prime number to reduce collisions.
3. **Collisions**
   A collision occurs when two keys hash to the same index. This is inevitable in most hashing systems and must be handled using specific techniques.

Types of Hashing

1. Static Hashing

- The size of the hash table remains fixed throughout.
- Common in applications where the data size is known in advance. □ Example: Hashing used in static dictionaries.

2. Dynamic Hashing

Date: 01-11-2026

- The hash table grows or shrinks as the number of keys changes.
- Suitable for applications with unpredictable or large datasets.
- Example: Hashing in databases to accommodate dynamic records.

```
# Simple hash function def
simple_hash(key, table_size):
return key % table_size

# Hash Table implementation with chaining
class HashTable:     def __init__(self, size):
self.table = [[] for _ in range(size)]

   def insert(self, key, value):
      index = simple_hash(key, len(self.table))        for pair in
self.table[index]:            if pair[0] == key:               pair[1] =
value  # Update value if key already exists               return
      self.table[index].append([key, value])  # Add new key-value pair

   def search(self, key):
      index = simple_hash(key, len(self.table))
for pair in self.table[index]:            if pair[0]
== key:               return pair[1]
      return None  # Key not found

   def delete(self, key):        index =
simple_hash(key, len(self.table))
      self.table[index] = [pair for pair in self.table[index] if pair[0] != key]

# Example usage hash_table =
HashTable(10)
hash_table.insert(15, "Value1")
hash_table.insert(25, "Value2")
print(hash_table.search(15))  # Output: Value1 hash_table.delete(15)
print(hash_table.search(15))  # Output: None
```

Muhammad Rehan 25F-CY-102

Date: 01-11-2026

Exercise:

1. Write a hash function and calculate hash values for the provided keys: 12, 25, 36, 48, 59, 63, 77, 84, 92, 101.
2. Create a hash table of size 10, insert the keys and their corresponding values ("A", "B", ..., "J") using the hash function, and store them at their respective indices.
3. Write a program to retrieve and display the values for the keys: 36, 63, and 101 from the hash table.
4. Display the entire hash table after inserting all keys and values, showing each index and its stored data.

Date: 01-11-2026

Introduction:

Hashing is a technique used to organize data in a way that makes searching and storing information faster. It uses a hash function to convert a key into a specific index of a hash table. When two keys produce the same index, a collision occurs. Chaining is one method used to manage such collisions.

Abstract:

This experiment focuses on implementing hashing techniques in Python. A hash function based on the modulo operator is used to generate table indices. Different keys are inserted along with their corresponding values into a hash table of size 10. The experiment also includes searching for specific keys and managing collisions through chaining.

Procedure:

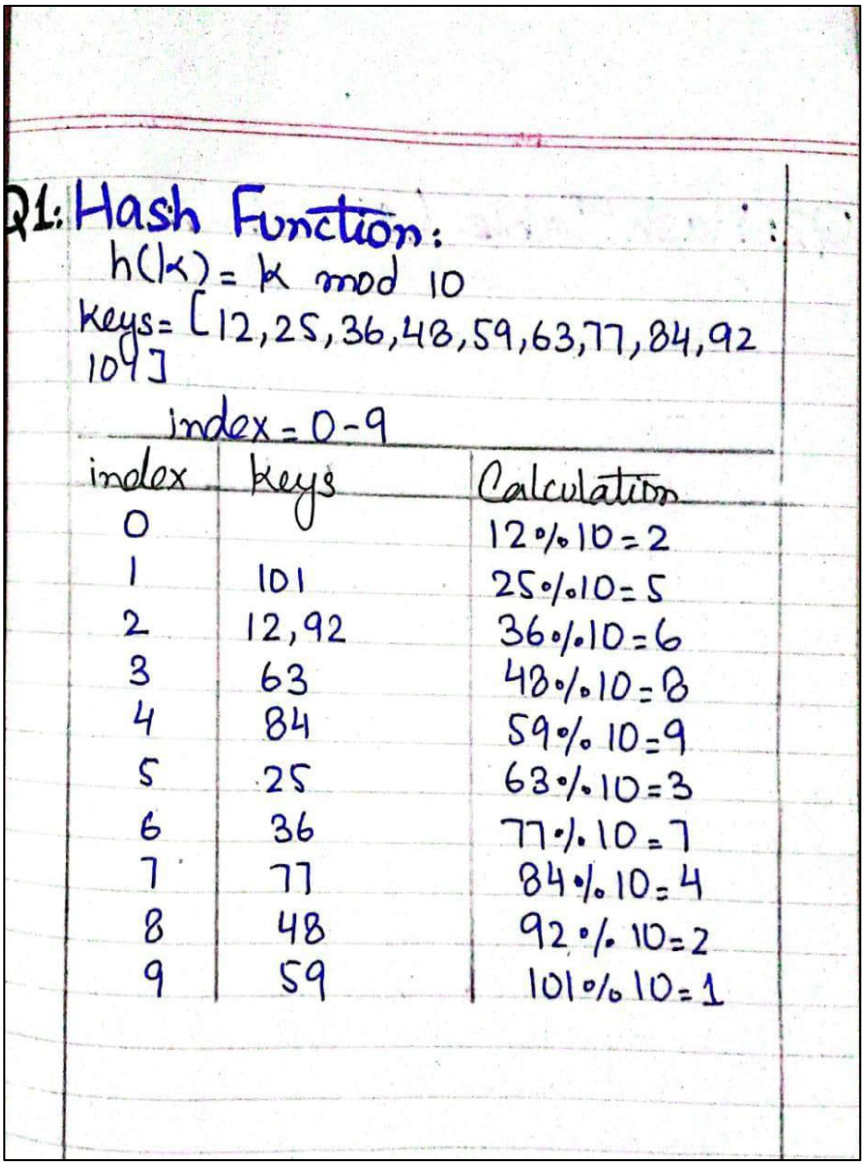
1. Reviewed the basic concepts of hashing, hash functions, and hash tables.
2. Created a new notebook in Google Colab for writing and executing the Python programs.
3. Defined a hash function using the modulo (%) operator.
4. Computed the hash indices for all the given keys.
5. Initialized a hash table with a capacity of 10.
6. Inserted the given keys along with their assigned values (A to J).
7. Applied the chaining technique to manage keys that produced identical indices.
8. Used the search operation to find the values associated with keys 36, 63, and 101.
9. Printed the complete hash table to view all indices and their stored elements.
10. Executed the programs and verified the results.

Conclusion:

Through this practical, we explored how hashing and hash tables work in Python. We created a hash function with the modulo operator and stored different key-value pairs in a hash table. We also tested the searching process and managed duplicate indices with the help of chaining. Overall, the experiment demonstrated how hashing makes data organization and retrieval more efficient.

Date: 01-11-2026

1. Write a hash function and calculate hash values for the provided keys: 12, 25, 36, 48, 59, 63, 77, 84, 92, 101.



Q1: Hash Function:

h(k) = k mod 10

keys = [12, 25, 36, 48, 59, 63, 77, 84, 92, 101]

index = 0-9

index	keys	Calculation
0		12%10=2
1	101	25%10=5
2	12, 92	36%10=6
3	63	48%10=8
4	84	59%10=9
5	25	63%10=3
6	36	77%10=7
7	77	84%10=4
8	48	92%10=2
9	59	101%10=1

Date: 01-11-2026

2. Create a hash table of size 10, insert the keys and their corresponding values ("A", "B", ..., "J") using the hash function, and store them at their respective indices.

Q2: Hash Table (size 10):

Keys = [12,25,36,48,59,63,77,84,92, 101]

Values = [A,B,C,D,E,F,G,H,I,J]

index = 0-9

index	keys	Values
0		
1	101	J
2	12,92	A,I
3	63	F
4	84	H
5	25	B
6	36	C
7	77	G
8	48	D
9	59	E

Collision at index 2 (12 and 92), handled by chaining

Date: 01-11-2026

3. Write a program to retrieve and display the values for the keys: 36, 63, and 101 from the hash table.

```python
def simple_hash(key, table_size):
    return key % table_size


class HashTable:
    def __init__(self, size):
        self.table = [[] for _ in range(size)]

    def insert(self, key, value):
        index = simple_hash(key, len(self.table))
        self.table[index].append([key, value])

    def search(self, key):
        index = simple_hash(key, len(self.table))

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

hash_table = HashTable(10)

keys = [12, 25, 36, 48, 59, 63, 77, 84, 92, 101]
values = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]

for key, value in zip(keys, values):
    hash_table.insert(key, value)

for key in [36, 63, 101]:
    print("Value of", key, ":", hash_table.search(key))
```

```
Value of 36 : C
Value of 63 : F
Value of 101 : J
```

Date: 01-11-2026

4. Display the entire hash table after inserting all keys and values, showing each index and its stored data.

```python
def simple_hash(key, table_size):
    return key % table_size

class HashTable:
    def __init__(self, size):
        self.table = [[] for _ in range(size)]

    def insert(self, key, value):
        index = simple_hash(key, len(self.table))
        self.table[index].append([key, value])

    def display(self):
        for index in range(len(self.table)):
            print(index, ":", self.table[index])


hash_table = HashTable(10)

keys = [12, 25, 36, 48, 59, 63, 77, 84, 92, 101]
values = ["A", "B", "C", "D", "E",
          "F", "G", "H", "I", "J"]

for key, value in zip(keys, values):
    hash_table.insert(key, value)

hash_table.display()
```

```
0 : []
1 : [[101, 'J']]
2 : [[12, 'A'], [92, 'I']]
3 : [[63, 'F']]
4 : [[84, 'H']]
5 : [[25, 'B']]
6 : [[36, 'C']]
7 : [[77, 'G']]
8 : [[48, 'D']]
9 : [[59, 'E']]
```